

SOLUÇÃO

(POSCOMP 2004 - Questão 39) Em um sistema operacional, um processo pode, em um dado instante de tempo, estar em um de três estados: em execução, pronto ou bloqueado. Considere as afirmativas abaixo sobre as possíveis transições entre estes estados que um processo pode realizar.

- I. Do estado em *execução* para o estado *bloqueado*
- II. Do estado em *execução* para o estado *pronto*
- III. Do estado *pronto* para o estado em *execução*
- IV. Do estado *pronto* para o estado *bloqueado*
- V. Do estado *bloqueado* para o estado em *execução*
- VI. Do estado *bloqueado* para o estado *pronto*

- | | |
|----------------------------|--|
| ☐ A | Somente as afirmativas I, II e III são verdadeiras. |
| ☐ B | Somente as afirmativas I, III, IV e V são verdadeiras. |
| ☐ C | Todas as afirmativas são verdadeiras. |

- | | |
|---------------------------------------|---|
| ☐ D | Somente as afirmativas I, III, IV e VI são verdadeiras. |
| ☒ E | Somente as afirmativas I, II, III e VI são verdadeiras. |

(POSCOMP 2014 - Questão 50) Em relação ao gerenciamento de processos, atribua V (verdadeiro) ou F (falso) às afirmativas a seguir.

- () Na espera ocupada, o processo é transferido para estado de bloqueado até que sua fatia de tempo termine e então ele retorna para fila de prontos.
- () O bloco de controle de processos (BCP – *Process Control Block*) é utilizado para armazenar informações sobre processos, e essas informações são utilizadas na troca de contexto de processos.
- () *Threads* apresentam menor custo de criação quando comparadas aos processos, pois compartilham alguns elementos do processo, como espaço de endereçamento.
- () Um processo pode estar nos seguintes estados: pronto, aguardando execução, em execução e bloqueado.
- () Um processo pode ser criado por uma chamada de sistema `fork()`, nesse caso, o processo gerado (conhecido como “filho”) é uma cópia exata do processo original, com os mesmos valores de variáveis em memória, diferenciando-se apenas no identificador do processo. Assinale a alternativa que contém, de cima para baixo, a sequência correta.

- | | |
|---------------------------------------|----------------|
| ☐ A | F, F, F, V, V. |
| ☒ B | F, V, V, F, V. |

- | | |
|----------------------------|----------------|
| ☐ C | V, F, F, F, V. |
| ☐ D | V, V, F, V, F. |

- | | |
|----------------------------|----------------|
| ☐ E | V, F, V, F, F. |
|----------------------------|----------------|

SOLUÇÃO

(POSCOMP 2019 - Questão 45) Considere o programa abaixo escrito em linguagem C. No instante da execução da linha 5, ter-se-á uma hierarquia composta de quantos processos e threads, respectivamente?

```
main() {
    int i;

    for (i=0; i<3; i++)
        fork();

    while(1);
}
```

- | | | | | |
|--|---------------------------------|---------------------------------|---------------------------------|---------------------------------|
| ☒ 8 e 8. | ☐ 1 e 0. | ☐ 4 e 1. | ☐ 7 e 7. | ☐ 3 e 0. |
|--|---------------------------------|---------------------------------|---------------------------------|---------------------------------|

(POSCOMP 2022 - Questão 46) O programa (em linguagem C) abaixo executa em um sistema operacional da família UNIX. Considere que todas as rotinas invocadas no programa executam sem erro. Assinale a alternativa que indica o resultado impresso na tela pelo programa.

```
signed int i;
int main(void) {
    if (fork() > 0)
        i++;
    else
        i++;
    i++;
    printf("%d\n", i);
}
```

- | | |
|---|--|
| ☐ 1 1
☒ 2 2
☐ 3 3 | ☐ Indeterminado Indeterminado

☐ 4 4 |
|---|--|

(POSCOMP 2025 - Questão 42) Tipicamente, a execução de um processo em um sistema operacional de tempo compartilhado caracteriza-se pela alternância entre ciclos de execução da CPU e ciclos de espera de E/S, até o seu término. Os tempos médios dos ciclos de execução da CPU e de espera de E/S denominaremos, respectivamente, `exec_t` e `es_t`. Considere um conjunto de processos que seguem esse padrão de alternância entre CPU e E/S, os quais executam sob um sistema operacional com escalonamento do tipo round-robin (RR) preemptivo. Assuma que o tempo médio para realizar uma troca de contexto seja dado por `switch_t`. Assinale a alternativa que melhor minimiza o tempo médio de turnaround dos processos.

- | | |
|---|---|
| ☐ (<code>exec_t > switch_t</code>) e (<code>exec_t > quantum</code>).
☒ (<code>exec_t > switch_t</code>) e (<code>exec_t < quantum</code>).
☐ (<code>exec_t < switch_t</code>) e (<code>exec_t < quantum</code>). | ☐ (<code>exec_t + switch_t > quantum</code>).

☐ (<code>exec_t < switch_t</code>) e (<code>exec_t > quantum</code>). |
|---|---|

SOLUÇÃO

(PCS3746 - 2025) Considere o código em Assembly abaixo que utiliza `fork` para criar dois processo. Assuma que o código compila e executa sem gerar erros ou exceções em tempo de execução.

```
section .data
char_A db 'A', 0
char_B db 'B', 0

section .text
global _start

_start:
;Instrucoes para realizar o fork()
mov eax, 2
int 0x80

cmp eax, 0
jnz B ;Se EAX for != 0, salta para B:
jmp finalizar
;
;
;
;
;
;
```

```
A:
; Instrucoes para mostrar char_A
mov ebx, 1
mov ecx, char_A
mov edx, 3
mov eax, 4
int 0x80

jmp finalizar

B:
; Instrucoes para mostrar char_B
mov ebx, 1
mov ecx, char_B
mov edx, 3
mov eax, 4
int 0x80

finalizar:
mov eax, 1
xor ebx, ebx
int 0x80
```

Assinale todas as alternativas **verdadeiras**.

- ☐ A A execução desse código exibirá os caracteres AB ou BA dependendo do escalonamento feito pelo sistema operacional.
- ☐ B O processo filho executará o trecho de código do rótulo B.
- ☒ C Os processo pai e filho executam o trecho de código do rótulo `finalizar`.
- ☐ D Se o sistema operacional utilizar um algoritmo de escalonamento do tipo não-preemptivo, a execução do código exibirá os caracteres AB ou BA.
- ☐ E Se o sistema operacional utilizar o algoritmo de escalonamento do tipo alternância circular (*Round-Robin*), a execução do código exibirá os caracteres AB ou BA.
- ☐ F O processo filho executará o trecho de código do rótulo A.
- ☒ G O único processo que executará o trecho de código do rótulo B será o processo pai.

SOLUÇÃO

(PCS3746 - 2025) Assuma um cenário hipotético em que um sistema operacional **não garante** que os registradores de propósito geral são salvos e recuperados no início e término da rotina de tratamento de interrupções, respectivamente. Assinale a solução que deveria ser implementada para que o tratamento de interrupções funcione corretamente no sentido de garantir que a execução dos processos não seja afetada.

- ☒ Utilizar as instruções **pusha** e **popa** no início e fim (antes de **iret**), respectivamente, da(s) rotina(s) de tratamento de interrupções.
- ☐ Utilizar as instruções **cli** e **sti** para desabilitar e habilitar interrupções enquanto o tratamento de uma interrupção é executado.
- ☐ Utilizar a instrução **xchg** para garantir atomicidade entre as operações dentro da rotina de tratamento de interrupções.
- ☐ Aumentar o intervalo em que as interrupções ocorrem.
- ☐ Utilizar as instruções **pushf** e **pop** no início e fim (antes de **iret**), respectivamente, da(s) rotina(s) de tratamento de interrupções.
- ☐ Mover o registrador de segmento de código (**CS**) para outra posição de memória.

(PCS3746 - 2025) No exemplo visto em aula de interrupções no DoS (utilizando o DosBox), o endereço de tratamento de interrupções do tipo **int 21h** estava localizado na posição de memória 0×0084 , onde os dois primeiros bytes indicavam o endereço da rotina e os dois últimos bytes o segmento de código. Suponha que um sistema operacional implementará uma interrupção denominada **int 4h**. Seguindo o mesmo raciocínio adotado para a **int 21h**, para que o sistema operacional saiba quais instruções executar ao tratar uma interrupção do tipo **int 4h**, em qual posição de memória ser armazenado o endereço da rotina que trata a interrupção **int 4h**? Observação importante: nas alternativas desta questão ignore aspectos relacionados ao segmento de código (**CS**) e o formato de armazenamento *little-endian*.

- ☐ 0×0084 | ☐ 0×0000 | ☐ 0×0016 | ☒ 0×0010 | ☐ 0×0085

Considere a representação conceitual de endereço de memória $\times$ conteúdo (instrução) abaixo. Nesta representação, todos os endereços estão em **hexadecimal** (o prefixo $0 \times$ foi omitido por simplicidade).

0106	pusha	010C	pusha	0112	pusha	0118	pusha	011E	pusha
0107	pushf	010D	pushf	0113	pushf	0119	pushf	011F	pushf
0108	add esp, 8	010E	add esp, 4	0114	sub esp, 8	011A	pop rax	0120	pop eax
0109	add esp, 2	010F	nop	0115	sub esp, 2	011B	nop	0121	pop ebx
010A	popa	0110	popa	0116	popa	011C	popa	0122	nop
010B	iret	0111	iret	0117	iret	011D	iret	0123	iret

A partir dessa memória e considerando uma **arquitetura de 32 bits**, qual o conteúdo deve ser colocado no endereço da **int 4h** (respondido na questão anterior) para que a rotina de tratamento desta interrupção tenha o seguinte comportamento: salvar registradores de propósito geral, salvar as flags de estado da CPU e recuperar o conteúdo dos registradores de propósito geral antes de retornar da interrupção.

- ☐ 0×0112 | ☐ 0×0106 | ☒ $0 \times 010C$ | ☐ 0×0118 | ☐ $0 \times 011E$